

Java Collections for Beginners (Tier 1)

A practical guide to the most common collection types in Java. We will cover the basics every Java developer needs, with plenty of examples.

This Tier 1 guide focuses on the three main collection types you will use every day: `ArrayList`, `HashMap`, and `HashSet`.

What are Collections?

A collection is a container that holds a group of objects. Java gives you many ready-made collection classes so you do not have to build them yourself.

Why not just use arrays?

- Arrays have a fixed size. Collections can grow and shrink.
- Arrays only have a few built-in operations. Collections have many useful methods.
- Collections come in different types for different needs (ordered, unique, key-value, etc.)

Here is a quick comparison:

```
java

// Array: fixed size
int[] arr = new int[5];
arr[0] = 10;

// ArrayList: grows automatically
List<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
list.add(30);
```

The Collection Framework Overview

Java Collections are organized into a hierarchy. At the top level, there are three main types you will see most often.

```
Collection (interface)
├── List    : ordered, allows duplicates
├── Set     : no duplicates
└── Queue  : ordered for processing (FIFO, LIFO)

Map (interface, separate from Collection)
└── Stores key-value pairs
```

The three most common implementations:

Interface	Most Common Implementation	Use For
List	ArrayList	Ordered list of items
Set	HashSet	Unique items, no order needed
Map	HashMap	Key-value lookup

In Tier 1, we focus on these three. They cover most everyday use cases.

1. ArrayList : An Ordered List

`ArrayList` is the most commonly used collection in Java. It is an ordered list of items. It allows duplicates. It can grow and shrink as you add or remove items.

Creating an ArrayList

```
java

import java.util.ArrayList;
import java.util.List;

// Empty list
List<String> names = new ArrayList<>();

// With initial values
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3));

// You can also use ArrayList directly as the type
ArrayList<String> cities = new ArrayList<>();
```

It is good practice to declare the variable as `List<T>` (the interface) and create it as `new ArrayList<>()` (the implementation). This way, you can swap implementations later without changing the rest of your code.

Adding elements

```
java

List<String> names = new ArrayList<>();

names.add("Alice");           // adds to the end
names.add("Bob");
names.add("Carol");

names.add(1, "Dan");          // inserts at index 1

System.out.println(names);    // [Alice, Dan, Bob, Carol]
```

Getting an element

```
java

String first = names.get(0);   // "Alice"
String second = names.get(1);  // "Dan"
```

If the index is out of range, you get an `IndexOutOfBoundsException`.

Updating an element

```
java

names.set(0, "Alex");          // replace at index 0
System.out.println(names);     // [Alex, Dan, Bob, Carol]
```

Removing an element

```
java

names.remove(0);               // remove by index
names.remove("Bob");           // remove by value (first match)
```

Be careful with primitive types. `list.remove(2)` removes by index, `list.remove(Integer.valueOf(2))` removes the value 2.

```
java

List<Integer> nums = new ArrayList<>(List.of(10, 20, 30));

nums.remove(1);                // removes element at index 1, list is [10, 30]
nums.remove(Integer.valueOf(10)); // removes the value 10, list is [30]
```

Checking size and emptiness

```
java

names.size();           // number of items
names.isEmpty();        // true if size is 0
```

Checking if an item exists

```
java

names.contains("Alice"); // true or false
names.indexOf("Bob");     // returns index, or -1 if not found
```

Clearing the list

```
java

names.clear(); // removes all items
```

Common ArrayList example

```
java

List<String> shoppingList = new ArrayList<>();

shoppingList.add("milk");
shoppingList.add("bread");
shoppingList.add("eggs");

if (!shoppingList.contains("butter")) {
    shoppingList.add("butter");
}

shoppingList.remove("bread");

System.out.println("Items: " + shoppingList.size());
for (String item : shoppingList) {
    System.out.println("- " + item);
}
```

When to use ArrayList

- You need an ordered collection
- Duplicates are okay
- You access elements by index a lot

- You usually add to the end of the list

ArrayList is the default choice for a list. If you do not have a specific reason to choose something else, use ArrayList.

2. HashMap : Key-Value Pairs

HashMap stores pairs of keys and values. You give it a key, it gives you back the value. Great for fast lookups.

Creating a HashMap

```
java

import java.util.HashMap;
import java.util.Map;

Map<String, Integer> ages = new HashMap<>();

ages.put("Alice", 25);
ages.put("Bob", 30);
ages.put("Carol", 28);
```

The first type is the key type, the second is the value type. Here we have String keys and Integer values.

Getting a value

```
java

Integer aliceAge = ages.get("Alice");    // 25
Integer unknownAge = ages.get("Unknown"); // null
```

If the key does not exist, you get null.

Default values

To avoid null, use getOrDefault.

```
java

int age = ages.getOrDefault("Unknown", 0); // 0
int aliceAge = ages.getOrDefault("Alice", 0); // 25
```

Adding or updating

put adds a new entry or replaces an existing one.

```
java
```

```
ages.put("Alice", 26);    // updates Alice's age to 26
ages.put("Dan", 22);      // adds Dan
```

Adding only if not already present

```
java
```

```
ages.putIfAbsent("Alice", 100); // does nothing, Alice already exists
ages.putIfAbsent("Eve", 35);    // adds Eve
```

Removing an entry

```
java
```

```
ages.remove("Bob"); // removes Bob
```

Checking if a key or value exists

```
java
```

```
ages.containsKey("Alice"); // true
ages.containsValue(28);    // true (Carol's age)
```

`containsKey` is fast. `containsValue` is slow because it scans every entry.

Size

```
java
```

```
ages.size(); // number of entries
ages.isEmpty(); // true if no entries
```

Getting all keys, values, or entries

```
java
```

```
ages.keySet(); // a Set of all keys
ages.values(); // a Collection of all values
ages.entrySet(); // a Set of all key-value pairs
```

Iterating a HashMap

The cleanest way is `forEach`.

```
java
```

```
ages.forEach((name, age) -> {  
    System.out.println(name + " is " + age);  
});
```

You can also use entry sets.

```
java  
  
for (Map.Entry<String, Integer> entry : ages.entrySet()) {  
    System.out.println(entry.getKey() + " is " + entry.getValue());  
}
```

Or just keys.

```
java  
  
for (String name : ages.keySet()) {  
    System.out.println(name + " is " + ages.get(name));  
}
```

Important: HashMap has no guaranteed order

```
java  
  
Map<String, Integer> ages = new HashMap<>();  
ages.put("Alice", 25);  
ages.put("Bob", 30);  
ages.put("Carol", 28);  
  
System.out.println(ages);  
// Could print in any order:  
// {Bob=30, Alice=25, Carol=28}
```

If you need to keep insertion order, use `LinkedHashMap` instead. We will cover that in a later tier.

Common HashMap example: counting words

```
java
```

```
String text = "the quick brown fox jumps over the lazy dog the fox";
String[] words = text.split(" ");

Map<String, Integer> counts = new HashMap<>();

for (String word : words) {
    counts.put(word, counts.getDefault(word, 0) + 1);
}

System.out.println(counts);
// {the=3, fox=2, quick=1, brown=1, jumps=1, over=1, lazy=1, dog=1}
```

This is a classic interview question.

When to use HashMap

- You need to look up values by some key
- You do not care about order
- You want fast access (most operations are very fast)

HashMap is the default choice for key-value storage.

3. HashSet : Unique Items

`HashSet` stores unique items. It does not allow duplicates. It does not keep any order.

Creating a HashSet

```
java

import java.util.HashSet;
import java.util.Set;

Set<String> names = new HashSet<>();
```

Adding elements

```
java

names.add("Alice");
names.add("Bob");
names.add("Alice");    // duplicate, ignored

System.out.println(names.size());    // 2
```

`add` returns `true` if the item was new, `false` if it was already there.

```
java

boolean added = names.add("Carol");    // true
boolean added2 = names.add("Carol");    // false
```

This is useful for checking if you have seen something before.

Checking if an item exists

```
java

names.contains("Alice");    // true
names.contains("Eve");      // false
```

This is one of the main reasons to use a Set. Checking if an item exists is very fast.

Removing an element

```
java

names.remove("Alice");
```

Size

```
java

names.size();
names.isEmpty();
```

Iterating

```
java

for (String name : names) {
    System.out.println(name);
}

// Or with forEach
names.forEach(System.out::println);
```

Important: HashSet has no order

```
java
```

```
Set<Integer> nums = new HashSet<>();
nums.add(3);
nums.add(1);
nums.add(2);

System.out.println(nums);
// Could print [1, 2, 3] or [3, 1, 2] or any order
```

If you need order, use `LinkedHashSet` (insertion order) or `TreeSet` (sorted order). We will cover those in a later tier.

Common HashSet example: removing duplicates

```
java

List<Integer> nums = List.of(1, 2, 2, 3, 3, 3, 4);

Set<Integer> unique = new HashSet<>(nums);
System.out.println(unique); // [1, 2, 3, 4] (some order)
```

Common HashSet example: tracking what you have seen

```
java

List<String> words = List.of("apple", "banana", "apple", "cherry", "banana");

Set<String> seen = new HashSet<>();
List<String> firstOccurrences = new ArrayList<>();

for (String word : words) {
    if (seen.add(word)) { // returns true if new
        firstOccurrences.add(word);
    }
}

System.out.println(firstOccurrences);
// [apple, banana, cherry]
```

When to use HashSet

- You need a collection of unique items
- You do not care about order
- You want very fast `contains` checks

A Set is also great when you need to compare two collections (intersection, union, difference).

Iterating Collections

There are several ways to loop through a collection. Here are the most common.

For-each loop (most common)

```
java

List<String> names = List.of("Alice", "Bob", "Carol");

for (String name : names) {
    System.out.println(name);
}
```

Works for any List, Set, or array.

forEach method (Java 8+)

```
java

names.forEach(name -> System.out.println(name));

// Or with method reference
names.forEach(System.out::println);
```

Traditional for loop (when you need the index)

```
java

for (int i = 0; i < names.size(); i++) {
    System.out.println(i + ": " + names.get(i));
}
```

This works for List but not for Set (Sets do not have indexes).

Iterator (when you need to remove during iteration)

```
java

Iterator<String> it = names.iterator();
while (it.hasNext()) {
    String name = it.next();
    if (name.startsWith("A")) {
        it.remove();    // safe way to remove
    }
}
```

If you try to remove using the list directly while iterating, you get a `ConcurrentModificationException`.

removeIf (Java 8+, often the cleanest)

```
java  
  
names.removeIf(name -> name.startsWith("A"));
```

Much cleaner than the iterator approach.

Choosing the Right Collection

This is asked in interviews all the time. Here is a simple decision guide.

Do you need key-value pairs?

Yes, use a `Map`. Default to `HashMap`.

Do you only care about unique items?

Yes, use a `Set`. Default to `HashSet`.

Do you need ordered items, possibly with duplicates?

Yes, use a `List`. Default to `ArrayList`.

Quick chart

Need	Use
Ordered list, by index	<code>ArrayList</code>
Unique items, fast contains check	<code>HashSet</code>
Key-value lookup	<code>HashMap</code>

For 80% of cases, one of these three is the right answer.

Putting It All Together

Real examples that use multiple collections.

Example 1: Group items by category

```
java
```

```
List<String> items = List.of("apple", "banana", "carrot", "broccoli", "avocado");

Map<Character, List<String>> byLetter = new HashMap<>();

for (String item : items) {
    char letter = item.charAt(0);
    byLetter.putIfAbsent(letter, new ArrayList<>());
    byLetter.get(letter).add(item);
}

System.out.println(byLetter);
// {a=[apple, avocado], b=[banana, broccoli], c=[carrot]}
```

Example 2: Find duplicates

```
java

List<Integer> nums = List.of(1, 2, 3, 2, 4, 5, 3, 6);

Set<Integer> seen = new HashSet<>();
Set<Integer> duplicates = new HashSet<>();

for (Integer n : nums) {
    if (!seen.add(n)) {
        duplicates.add(n);
    }
}

System.out.println(duplicates); // [2, 3]
```

Example 3: Count items

```
java

List<String> votes = List.of("yes", "no", "yes", "yes", "no", "abstain");

Map<String, Integer> counts = new HashMap<>();
for (String vote : votes) {
    counts.put(vote, counts.getOrDefault(vote, 0) + 1);
}

System.out.println(counts); // {yes=3, no=2, abstain=1}
```

Example 4: Find common items between two lists

```
java
```

```
List<String> teamA = List.of("Alice", "Bob", "Carol");
List<String> teamB = List.of("Bob", "Dan", "Carol");

Set<String> setA = new HashSet<>(teamA);
Set<String> setB = new HashSet<>(teamB);

setA.retainAll(setB); // keeps only items in both
System.out.println(setA); // [Bob, Carol]
```

Example 5: A simple phone book

```
java

Map<String, String> phoneBook = new HashMap<>();
phoneBook.put("Alice", "555-1111");
phoneBook.put("Bob", "555-2222");
phoneBook.put("Carol", "555-3333");

String name = "Bob";
String phone = phoneBook.getOrDefault(name, "Not found");
System.out.println(name + ": " + phone);

// List all contacts
phoneBook.forEach((n, p) -> System.out.println(n + " -> " + p));
```

Example 6: Track unique visitors

```
java

Set<String> visitors = new HashSet<>();

String[] visits = {"alice@x.com", "bob@x.com", "alice@x.com", "carol@x.com"};

for (String email : visits) {
    visitors.add(email);
}

System.out.println("Unique visitors: " + visitors.size()); // 3
```

Common Mistakes

Mistake 1: Using == instead of equals to compare items

```
java
```

```
List<String> names = new ArrayList<>();
names.add("Alice");

String s = new String("Alice");
names.contains(s);    // true (uses equals, not ==)

// But this is wrong:
if (names.get(0) == s) {    // false, compares references
    // ...
}
```

`contains()`, `indexOf()`, and similar methods use `equals()` to compare. Always use `equals()` in your own code too.

Mistake 2: Confusing `remove(int)` with `remove(Object)`

```
java

List<Integer> nums = new ArrayList<>(List.of(10, 20, 30));

nums.remove(1);    // removes index 1 (the value 20)
// nums is now [10, 30]
```

If you want to remove the value 1, you need to box it.

```
java

nums.remove(Integer.valueOf(1));
```

This is a classic interview trap.

Mistake 3: Modifying a collection while iterating

```
java

List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4, 5));

for (Integer n : nums) {
    if (n % 2 == 0) {
        nums.remove(n);    // ConcurrentModificationException
    }
}
```

Use `removeIf()` or an iterator instead.

```
java
```

```
nums.removeIf(n -> n % 2 == 0);
```

Mistake 4: Calling get on a HashMap and assuming non-null

```
java  
  
Map<String, Integer> map = new HashMap<>();  
int value = map.get("missing").intValue();    // NullPointerException
```

Use `getOrDefault` or check for null.

```
java  
  
int value = map.getOrDefault("missing", 0);
```

Mistake 5: Adding to an immutable collection

```
java  
  
List<Integer> nums = List.of(1, 2, 3);  
nums.add(4);    // UnsupportedOperationException
```

`List.of`, `Set.of`, and `Map.of` create immutable collections. To get a mutable one, wrap it.

```
java  
  
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3));  
nums.add(4);    // works
```

Mistake 6: Forgetting that HashSet and HashMap have no order

```
java  
  
Set<Integer> set = new HashSet<>();  
set.add(3);  
set.add(1);  
set.add(2);  
  
System.out.println(set);    // could be any order, do not rely on it
```

If you need order, use `LinkedHashSet` or `TreeSet`.

Mistake 7: Not using generics

```
java
```

```
// Bad: raw type
List names = new ArrayList();
names.add("Alice");
names.add(123); // accidentally added a number, no error

// Good: with generics
List<String> names = new ArrayList<>();
names.add("Alice");
// names.add(123); // compile error
```

Always use generics. They catch type errors at compile time.

Mistake 8: Wrong key types in HashMap

```
java

class User {
    String name;
    User(String name) { this.name = name; }
}

Map<User, Integer> ages = new HashMap<>();
ages.put(new User("Alice"), 25);

User u = new User("Alice");
ages.get(u); // returns null!
```

This is because `User` does not override `equals` and `hashCode`. The Map sees them as different objects.

If you use a class as a Map key or in a Set, override `equals` and `hashCode`. Or use a `record`, which does it automatically.

```
java

record User(String name) {} // equals and hashCode are automatic
```

Quick Cheat Sheet

ArrayList

Method	What it does
<code>add(item)</code>	Add to end

Method	What it does
<code>add(index, item)</code>	Insert at index
<code>get(index)</code>	Get item at index
<code>set(index, item)</code>	Replace item at index
<code>remove(index)</code>	Remove by index
<code>remove(item)</code>	Remove by value (first match)
<code>size()</code>	Number of items
<code>contains(item)</code>	Does it have this item?
<code>indexOf(item)</code>	Index of item, or -1
<code>isEmpty()</code>	Is it empty?
<code>clear()</code>	Remove everything

HashMap

Method	What it does
<code>put(key, value)</code>	Add or update entry
<code>get(key)</code>	Get value for key, or null
<code>getOrDefault(key, default)</code>	Get value, or default if missing
<code>putIfAbsent(key, value)</code>	Add only if key not present
<code>remove(key)</code>	Remove entry by key
<code>containsKey(key)</code>	Does this key exist?
<code>containsValue(value)</code>	Does this value exist? (slow)
<code>size()</code>	Number of entries
<code>keySet()</code>	All keys as a Set
<code>values()</code>	All values as a Collection
<code>entrySet()</code>	All key-value pairs
<code>forEach((k, v) -> ...)</code>	Loop through entries

HashSet

Method	What it does
<code>add(item)</code>	Add (returns false if duplicate)
<code>remove(item)</code>	Remove item
<code>contains(item)</code>	Does it have this item? (fast)
<code>size()</code>	Number of items
<code>isEmpty()</code>	Is it empty?
<code>clear()</code>	Remove everything

Final Tips

1. Default to `ArrayList` for a list, `HashMap` for key-value lookup, and `HashSet` for unique items. These three cover most needs.
2. Always use generics. `List<String>` is much safer than raw `List`.
3. Declare variables as the interface (`List`, `Map`, `Set`) and create as the implementation (`ArrayList`, `HashMap`, `HashSet`). This makes your code flexible.
4. Use `getOrDefault` instead of `get` when the key might be missing. Cleaner than null checks.
5. Use `removeIf` to remove items based on a condition. It is cleaner than manual iteration.
6. If you put your own class into a Set or use it as a Map key, override `equals` and `hashCode`. Or just use a record.
7. `HashMap` and `HashSet` do not preserve order. If you need order, look at `LinkedHashMap` and `LinkedHashSet`.
8. The collection methods `contains`, `indexOf`, etc. use `equals()`, not `==`. Make sure your classes have proper `equals` if you store them in collections.
9. Most operations on `ArrayList`, `HashMap`, and `HashSet` are very fast. They are designed for performance.
10. Practice the common interview patterns: counting items, finding duplicates, grouping by key, and finding intersections. They all use these basic collections.

You now have a strong base in Java collections. The next tier covers more specialized collections like `LinkedList`, `TreeMap`, `TreeSet`, `Queue`, and `Deque`, plus how to sort with `Comparable`

and `Comparator`.